

SMART STUDY PLANNER

Programming Fundamentals Coursework

Module Code : 1203 ST - Programming
Fundamentals

Student Name : Idola Richard

Reg NO : VU-BCD-2603-1014-DAY

LECTURER : Kinyonyi David Hope

Date : 06TH/09/2026

1. Introduction

The Smart Study Planner is a small console-based Python programme designed to help a student keep track of study sessions during a semester. I chose a simple menu approach so that the programme is easy to use without needing a graphical interface. A session records the subject, topic, date or day label and the number of minutes studied. The programme can also search the records and produce basic statistics.

Another important part of the project is saving the information to a file. This means the records are not lost when the programme is closed. When the programme starts again, it loads the saved sessions and continues from where the user stopped.

2. Objectives of the System

- Provide a menu-driven interface for managing study sessions.
- Allow the user to add sessions and validate the study duration.
- Classify sessions as Short, Medium or Long.
- Display all saved sessions in a readable table.
- Search for sessions using the subject name without case sensitivity.
- Calculate total study time and other useful statistics.
- Save sessions in study_log.txt and load them again when the programme starts.

3. Main Functions Used

Function	Purpose
main()	Controls the menu and keeps the programme running until the user selects Save and exit.
add_session()	Collects the subject, topic, date/day and duration, then stores the session as a dictionary.
classify_session(duration)	Returns Short, Medium or Long depending on the number of minutes.
view_sessions()	Shows every recorded session in a formatted table.
search_by_subject(subject)	Finds matching sessions without treating uppercase and lowercase as different.
study_statistics()	Calculates overall study hours, study time per subject, the least-studied subject and the longest session.
save_sessions()	Writes all sessions to study_log.txt.
load_sessions()	Loads saved sessions at startup and handles a missing or damaged file safely.

4. How the Program Works

Adding a study session

The user selects option 1 and enters the subject, topic and date/day. The programme then asks for the duration in minutes. A positive number is required. If the user enters text such as 'abc', zero, or a negative number, the programme asks again instead of crashing.

Classifying a session

The classification is handled by one function so it can be reused whenever a session is displayed. A session below 30 minutes is Short, 30 to 90 minutes is Medium, and anything above 90 minutes is Long.

Searching

The subject search converts both the saved subject and the typed search term to lower case before comparing them. Therefore, typing Python, python or PYTHON can find the same sessions.

Statistics

The programme first adds all session durations to get the overall study time. It then groups durations by subject. The smallest subject total is reported as the weakest area, and the session with the greatest duration is reported as the longest single session.

Saving and loading

The sessions are stored in study_log.txt using JSON format. JSON makes the file easy for Python to save and load while still meeting the required filename. On the first run, the file may not exist, so the programme catches FileNotFoundError and starts with an empty list.

5. Program Testing

I tested the programme using several normal and incorrect inputs. The following are the main checks:

Test	Input	Expected result
Invalid menu option	9	Displays an invalid-choice message and returns to the menu.
Invalid duration	abc	Rejects the value and asks for a number again.
Non-positive duration	0	Rejects the value and asks for a positive number.
Session classification	20 / 45 / 120 minutes	Returns Short / Medium / Long.

Case-insensitive search	python	Finds sessions saved under Python.
No search results	Networking	Shows a clear message instead of an empty table.
File persistence	Exit, then restart	Previously saved sessions are loaded again.

6. Example of a Test Run

SMART STUDY PLANNER

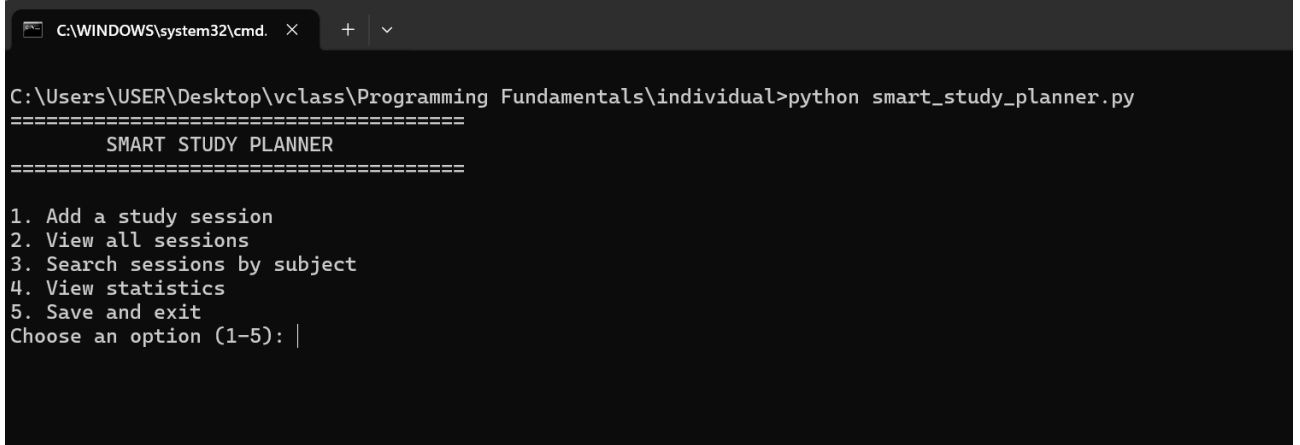
1. Add a study session
2. View all sessions
3. Search sessions by subject
4. View statistics
5. Save and exit

Choose an option (1-5): 2

No.	Subject	Topic	Minutes	Class
1	Python	Functions	45.0	Medium
2	Database Systems	SQL Basics	120.0	Long
3	Python	Loops	20.0	Short

7. Screenshot Checklist for the Answer Sheet

1. Screenshot 1: The main menu when the programme starts.

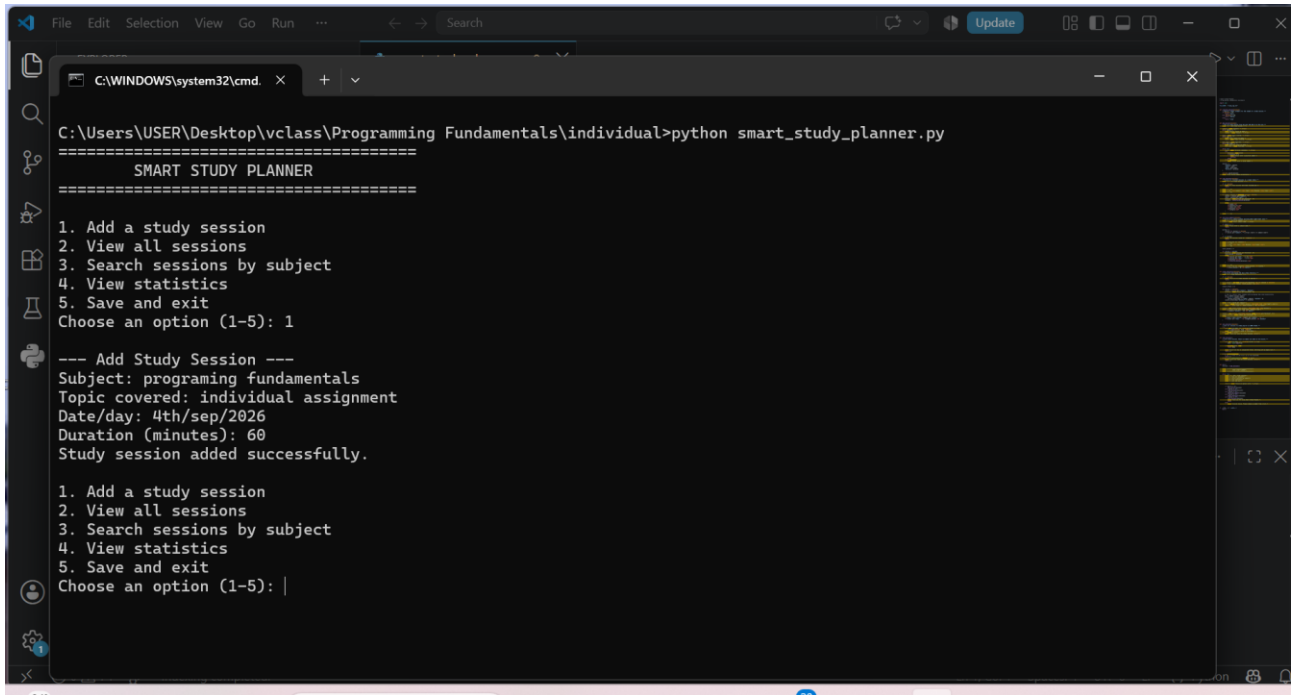


```
C:\WINDOWS\system32\cmd. X + v

C:\Users\USER\Desktop\vclass\Programming Fundamentals\individual>python smart_study_planner.py
=====
SMART STUDY PLANNER
=====

1. Add a study session
2. View all sessions
3. Search sessions by subject
4. View statistics
5. Save and exit
Choose an option (1-5): |
```

2. Screenshot 2: Adding a study session, including the duration validation message.



```
File Edit Selection View Go Run ... Search Update

C:\WINDOWS\system32\cmd. X + v

C:\Users\USER\Desktop\vclass\Programming Fundamentals\individual>python smart_study_planner.py
=====
SMART STUDY PLANNER
=====

1. Add a study session
2. View all sessions
3. Search sessions by subject
4. View statistics
5. Save and exit
Choose an option (1-5): 1

--- Add Study Session ---
Subject: programing fundamentals
Topic covered: individual assignment
Date/day: 4th/sep/2026
Duration (minutes): 60
Study session added successfully.

1. Add a study session
2. View all sessions
3. Search sessions by subject
4. View statistics
5. Save and exit
Choose an option (1-5): |
```

3. Screenshot 3: Viewing all study sessions.

```
1. Add a study session
2. View all sessions
3. Search sessions by subject
4. View statistics
5. Save and exit
Choose an option (1-5): 2
```

```
--- All Study Sessions ---
```

No.	Subject	Topic	Minutes	Class
1	2	2	20.0	Short
2	programing fundam	individual assignment	60.0	Medium

4. Screenshot 4: Searching for a subject and showing the total time.

```
1. Add a study session
2. View all sessions
3. Search sessions by subject
4. View statistics
5. Save and exit
Choose an option (1-5): 3
```

```
--- Search by Subject ---
```

```
Enter subject name: programing fundamentals
```

```
Sessions for programing fundamentals:
```

Date	Topic	Minutes	Class
4th/sep/2026	individual assignment	60.0	Medium

```
Total time for programing fundamentals: 60.0 minutes (1.00 hours)
```

5. Screenshot 5: Viewing the study statistics.

```
1. Add a study session
2. View all sessions
3. Search sessions by subject
4. View statistics
5. Save and exit
Choose an option (1-5): 4

--- Study Statistics ---
Total hours studied: 1.33

Time studied per subject:
- 2: 0.33 hours
- programing fundamentals: 1.00 hours

Subject with least study time: 2 (0.33 hours)
Longest single session: programing fundamentals - individual assignment (60.0 minutes)

1. Add a study session
```

6. Screenshot 6: Exiting and then restarting to show that saved sessions are loaded.

```
1. Add a study session
2. View all sessions
3. Search sessions by subject
4. View statistics
5. Save and exit
Choose an option (1-5): 5
Study sessions saved to study_log.txt.
Thank you for using Smart Study Planner.

C:\Users\USER\Desktop\vc\class\Programming Fundamentals\individual>
```

8. Conclusion

The Smart Study Planner meets the main requirements of the coursework. It uses separate functions, stores sessions in a list of dictionaries, validates input, classifies session length, supports subject searching, calculates statistics and keeps the records in a file between runs. The project also uses the standard Python entry-point check so that the main programme only runs when the file is executed directly.

GitHub link: